Laporan Program Kelompok 11:
Diffie-Hellman Key Exchange & ElGamal



Mata Kuliah: Kriptografi

Disusun oleh:

Kanafira Vanesha Putri	5027241010
Fika Arka Nuriyah	5027241071
S. Farhan Baig	5027241097

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Pada kriptografi simetri, pengirim dan penerima harus memiliki kunci rahasia yang sama. Masalahnya, bagaimana kunci itu dapat dibagikan melalui saluran komunikasi yang tidak aman tanpa diketahui penyadap? Whitfield Diffie dan Martin Hellman menjawab persoalan ini pada tahun 1976 melalui algoritma pertukaran kunci Diffie-Hellman. Taher ElGamal kemudian mengembangkan gagasan yang sama menjadi algoritma enkripsi kunci-publik pada tahun 1985. Keduanya mendapatkan keamanan dari sulitnya masalah **logaritma diskrit**.

Laporan ini mendokumentasikan program yang dibuat Kelompok 11 untuk mendemonstrasikan setiap tahapan Diffie-Hellman dan ElGamal. Seluruh algoritma diimplementasikan secara manual: **tidak memakai library atau framework kriptografi jadi**, sehingga setiap langkah perhitungan dapat ditelusuri dan dijelaskan.

1.2 Tujuan

- Mengimplementasikan pertukaran kunci Diffie-Hellman beserta fungsi pendukung (pembangkitan bilangan prima, pencarian akar primitif, eksponensiasi modular).
- Mengimplementasikan ElGamal: pembangkitan kunci, enkripsi, dan dekripsi untuk teks dan file.
- Menampilkan setiap tahapan perhitungan secara eksplisit agar mudah dipahami dan diverifikasi terhadap contoh pada materi Pertemuan 9.
- Menguji kebenaran program dengan contoh materi, parameter 256 bit, dan unit test.

1.3 Ruang Lingkup dan Batasan

- Bahasa pemrograman: Python 3 (hanya pustaka standar; tidak ada *pip install*).
- Operasi inti (mod_exp, gcd, invers modular, Miller-Rabin, akar primitif, pemecahan blok) ditulis sendiri. Sumber bilangan acak memakai `random.SystemRandom`.
- Program bersifat edukatif: ElGamal tanpa padding, dan cipher simetri pada demo kunci sesi hanya contoh sederhana (lihat Bab 7).

BAB 2 LANDASAN TEORI

2.1 Akar Primitif

Misalkan p bilangan prima. Bilangan g ($1 < g < p$) disebut **akar primitif** modulo p bila pangkat-pangkat g^1, g^2, …, g^{p-1} mod p menghasilkan seluruh bilangan 1 sampai p-1. Uji praktisnya: g adalah akar primitif jika untuk setiap faktor prima q dari p-1 berlaku:

$$g^{(p-1)/q} \bmod p \neq 1$$

Pada program, uji ini ada di `is_primitive_root`. Untuk p berukuran besar, p dibangkitkan sebagai **prima aman** $p = 2q + 1$ dengan q prima, sehingga faktor prima p-1 hanya $\{2, q\}$ dan uji akar primitif menjadi cepat tanpa memfaktorkan bilangan besar.

2.2 Logaritma Diskrit

Diberikan prima p, akar primitif g, dan $y = g^x \bmod p$, mencari x disebut masalah logaritma diskrit. Menghitung y dari x mudah (square-and-multiply), tetapi kebalikannya tidak diketahui memiliki algoritma efisien untuk p yang besar. Asimetri inilah dasar keamanan Diffie-Hellman dan ElGamal.

2.3 Pertukaran Kunci Diffie-Hellman

Alice (A) dan Bob (B) menyepakati bilangan prima n dan akar primitif g modulo n; keduanya **publik**. Tahapannya:

No.	Pelaku	Aksi	Rumus
1	A dan B	Menyepakati parameter publik	n prima, g akar primitif dari n
2	Alice	Memilih x acak (rahasia), mengirim X ke Bob	$X = g^x \bmod n$
3	Bob	Memilih y acak (rahasia), mengirim Y ke Alice	$Y = g^y \bmod n$
4	Alice	Menghitung kunci sesi	$K = Y^x \bmod n$
5	Bob	Menghitung kunci sesi	$K' = X^y \bmod n$

Karena $Y^x = (g^y)^x = g^{xy} = (g^x)^y = X^y \pmod n$, maka **K = K'**. Penyadap hanya mengetahui n, g, X, dan Y, tidak mengetahui x dan y, sehingga tidak dapat menghitung K tanpa menyelesaikan logaritma diskrit. K kemudian dipakai sebagai kunci sesi pada algoritma kriptografi simetri.

Contoh (materi): n = 251, g = 6. Alice memilih x = 124: $X = 6^{124} \bmod 251 = 209$. Bob memilih y = 158: $Y = 6^{158} \bmod 251 = 197$. Alice menghitung $K = 197^{124} \bmod 251 = 79$, Bob menghitung $K' = 209^{158} \bmod 251 = 79$. Terbukti K = K' = 79.

2.4 Algoritma ElGamal

a) Pembangkitan kunci

- Pilih bilangan prima p dan akar primitif g modulo p (boleh publik).
- Pilih kunci privat x dengan $1 < x < p$-1.
- Hitung $y = g^x \bmod p$.
- **Kunci publik** = (y, g, p); **kunci privat** = (x, p).

b) Enkripsi

Plainteks dipecah menjadi blok-blok bilangan m dengan $0 \le m \le p$-1. Untuk setiap blok, pilih bilangan acak k ($1 \le k \le p$-2) lalu hitung:

$$a = g^k \bmod p \qquad b = y^k \cdot m \bmod p$$

Cipherteks untuk blok tersebut adalah pasangan **(a, b)**; panjang cipherteks dua kali plainteks. Karena *k* acak, enkripsi pesan yang sama menghasilkan cipherteks berbeda.

c) Dekripsi

$$m = b \cdot (a^x)^{-1} \bmod p = b \cdot a^{p-1-x} \bmod p$$

Kesamaan kedua ruas mengikuti teorema kecil Fermat. Kebenaran: $b \cdot (a^x)^{-1} = y^k m \cdot g^{-kx} = g^{xk} m \cdot g^{-kx} = m \pmod p$.

Contoh (materi): p = 2273, g = 3, x = 243 menghasilkan $y = 3^{243} \bmod 2273 = 461$. Pesan 'HALO' dengan pemetaan A=00, …, Z=25 menjadi 07001114, dipecah menjadi $m_1 = 0700$ dan $m_2 = 1114$. Dengan k = 1463 diperoleh (a, b) = (1439, 74), dan dengan k = 2001 diperoleh (1220, 1682). Dekripsi mengembalikan 0700 dan 1114, yaitu 'HALO'.

BAB 3 STRUKTUR PROJECT

Project disusun modular: seluruh algoritma berada di paket `crypto/`, sedangkan program utama, demo, dan pengujian dipisahkan agar mudah dibaca.

```
kelompok11_dh_elgamal/
|-- main.py                 # menu interaktif (semua tahapan, tampilkan langkah)
|-- README.md
|-- crypto/
|   |-- __init__.py
|   |-- math_utils.py       # mod_exp, gcd, extended Euclid, invers, Miller-Rabin,
|   |                       # prima aman, akar primitif, logaritma diskrit brute force
|   |-- diffie_hellman.py   # parameter, nilai publik, kunci sesi, run_exchange
|   |-- elgamal.py          # generate_keypair, encrypt/decrypt blok, teks, byte/file
|   |-- encoding.py         # teks <-> blok bilangan (mode slide & mode byte)
|   |-- storage.py          # simpan/baca kunci dan cipherteks (format teks)
|   |-- session.py          # demo kunci sesi DH + cipher simetri XOR (edukasi)
|   `-- trace.py            # pencetak langkah-langkah perhitungan
|-- demo/
|   |-- demo_slide.py       # mereproduksi contoh materi (K=79, 'HALO')
|   `-- demo_lengkap.py     # demo otomatis 256 bit end-to-end
|-- tests/
|   `-- test_all.py         # 16 unit test (unittest bawaan Python)
|-- contoh/                 # contoh kunci, pesan, dan hasil enkripsi
`-- docs/
    |-- Laporan_Kelompok11.pdf
    `-- sampul_asli.pdf
```

Berkas	Peran
crypto/math_utils.py	Fondasi matematika: mod_exp, gcd, extended_gcd, mod_inverse, is_probable_prime, generate_prime, generate_safe_prime, prime_factors, is_primitive_root, find_primitive_root, discrete_log_bruteforce.
crypto/diffie_hellman.py	Tahapan Diffie-Hellman: generate_parameters, validate_parameters, generate_private_key, compute_public_value, compute_shared_secret, run_exchange.
crypto/elgamal.py	Tahapan ElGamal: generate_keypair, encrypt_block, decrypt_block, encrypt_bytes/decrypt_bytes (teks UTF-8 dan file), encrypt_text_slide/decrypt_text_slide.

Berkas	Peran
crypto/encoding.py	Konversi pesan ke blok bilangan < p: mode slide (A=00..Z=25, 4 digit) dan mode byte.
crypto/storage.py	Menyimpan dan membaca kunci publik/privat dan cipherteks dalam berkas teks.
crypto/session.py	Demo pemakaian kunci sesi DH pada cipher aliran XOR (hanya edukasi).
crypto/trace.py	Tracer yang mencetak tiap langkah perhitungan dengan nilai sebenarnya.
main.py	Menu interaktif untuk seluruh fitur.
demo/, tests/	Demo reproduksi materi dan demo otomatis; 16 unit test.

BAB 4 PERANCANGAN DAN IMPLEMENTASI

4.1 Fondasi Matematika (math_utils.py)

Eksponensiasi modular. Perhitungan g^x mod p dengan x ratusan bit tidak mungkin dilakukan dengan memangkatkan langsung. Fungsi `mod_exp` memakai metode *square-and-multiply*: eksponen ditelusuri bit demi bit sehingga hanya diperlukan O(log x) perkalian modular.

```
def mod_exp(base, exp, mod):
    if mod == 1:
        return 0
    if exp < 0:
        raise ValueError("eksponen harus >= 0")
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result
```

Invers modular. `extended_gcd` menghitung (g, s, t) dengan a·s + b·t = gcd(a, b); bila gcd = 1 maka s adalah invers a modulo b. Dipakai untuk memverifikasi hasil dekripsi $(a^x)^{-1}$.

Keprimaan dan prima aman. `is_probable_prime` menerapkan uji Miller-Rabin dengan 40 putaran (peluang salah < 4^{-40}). `generate_safe_prime` mencari *q* prima sehingga 2*q*+1 juga prima.

```
def generate_safe_prime(bits):
    while True:
        q = generate_prime(bits - 1)
        p = 2 * q + 1
        if p.bit_length() == bits and is_probable_prime(p):
            return p, q
```

Akar primitif. Mengikuti uji pada Bab 2.1:

```
def is_primitive_root(g, p, factors=None):
    if g <= 1 or g >= p:
        return False
    if factors is None:
        factors = prime_factors(p - 1)
    return all(mod_exp(g, (p - 1) // q, p) != 1 for q in factors)
```

4.2 Diffie-Hellman (diffie_hellman.py)

Setiap tahap protokol dipetakan ke satu fungsi sehingga dapat dipanggil dan ditunjukkan terpisah. Fungsi `run_exchange` menjalankan seluruh protokol dan mencetak setiap langkah lewat tracer.

Tahap	Fungsi
1. Parameter publik (n, g)	generate_parameters(bits) / validate_parameters(n, g)
2-3. Bilangan rahasia dan nilai publik	generate_private_key(n), compute_public_value(g, x, n)
4-5. Kunci sesi	compute_shared_secret(X_atau_Y, kunci_rahasia, n)
Seluruh protokol	run_exchange(n, g, x=None, y=None)

```
def compute_public_value(g, private, n):
    return mod_exp(g, private, n)

def compute_shared_secret(other_public, own_private, n):
    if not (1 < other_public < n - 1):
        raise ValueError("nilai publik pihak lain tidak valid")
    return mod_exp(other_public, own_private, n)
```

`compute_shared_secret` menolak nilai publik yang tidak valid (di luar 1 < X < n-1) untuk menghindari kunci sesi yang trivial.

4.3 ElGamal (elgamal.py)

Pembangkitan kunci memeriksa p prima, g akar primitif, 1 < x < p-1, kemudian menghitung $y = g^x \bmod p$. Jika p tidak diberikan, dibangkitkan prima aman sehingga faktor p-1 sudah diketahui.

```
def generate_keypair(p=None, g=None, x=None, bits=256, tr=SILENT):
    factors = None          # faktor prima dari p-1 (diketahui untuk prima aman)
    if p is None:
        p, q = generate_safe_prime(bits)
        factors = [2, q]
    else:
        if not is_probable_prime(p):
            raise ValueError("p harus bilangan prima")
    if g is None:
        g = find_primitive_root(p, factors=factors)
    elif not is_primitive_root(g, p, factors):
        raise ValueError("g bukan akar primitif dari p")

    if x is None:
        x = random_between(2, p - 2)
    elif not (1 < x < p - 1):
        raise ValueError("kunci privat x harus memenuhi 1 < x < p-1")

    y = mod_exp(g, x, p)
    return PublicKey(p, g, y), PrivateKey(p, g, x)
```

Enkripsi dan dekripsi satu blok. Rumus Bab 2.4 diterjemahkan langsung. Pada dekripsi, $(a^x)^{-1}$ dihitung dengan $a^{p-1-x} \bmod p$ (Fermat) sesuai materi, kemudian dibandingkan dengan invers dari extended Euclid sebagai verifikasi silang.

```
def encrypt_block(m, pub, k=None, tr=SILENT):
    p, g, y = pub
    if not (0 <= m <= p - 1):
        raise ValueError(f"blok m={m} di luar rentang [0, p-1]")
    if k is None:
        k = random_between(1, p - 2)
```

```
    elif not (1 <= k <= p - 2):
        raise ValueError("k harus pada rentang [1, p-2]")
    a = mod_exp(g, k, p)
    b = (mod_exp(y, k, p) * m) % p
    return a, b

def decrypt_block(a, b, priv, tr=SILENT):
    p, _, x = priv
    inv_fermat = mod_exp(a, p - 1 - x, p)
    m = (b * inv_fermat) % p
    # verifikasi silang dengan extended Euclid
    ax = mod_exp(a, x, p)
    inv_euclid = mod_inverse(ax, p)
    assert inv_fermat == inv_euclid, "invers Fermat != invers Euclid"
    return m
```

Pemecahan pesan menjadi blok (encoding.py)

Karena setiap blok *m* harus berada pada [0, p-1], pesan dipecah menjadi blok. Tersedia dua mode:

- **Mode slide**: huruf A-Z dipetakan A=00 … Z=25 lalu dipecah per 4 digit, persis seperti contoh materi ('HALO' → 07001114 → 0700, 1114).
- **Mode byte**: teks UTF-8 atau isi file dipecah per blok berukuran floor((jumlah bit p - 1) / 8) byte sehingga nilai blok selalu < p. Panjang data asli disimpan pada cipherteks agar blok terakhir yang lebih pendek (dan byte 0x00 di awal blok) dapat dipulihkan dengan benar. Mode ini juga dipakai untuk mengenkripsi file.

4.4 Penyimpanan dan Antarmuka

`storage.py` menyimpan kunci sebagai berkas teks sederhana (baris *nama=nilai*) dan cipherteks sebagai daftar baris *a:b* dengan header `ELGAMAL-v1` berisi mode, ukuran blok, dan panjang data. `main.py` menyediakan menu interaktif: Diffie-Hellman tahap demi tahap, pembangkitan kunci ElGamal, enkripsi/dekripsi teks, enkripsi/dekripsi file, demo contoh materi, dan demo serangan logaritma diskrit.

BAB 5 CARA MENJALANKAN PROGRAM

Kebutuhan hanya Python 3.8 atau lebih baru. Dari folder root project:

```
python main.py                              # menu interaktif
python demo/demo_slide.py                   # reproduksi contoh perhitungan materi
python demo/demo_lengkap.py                 # demo otomatis parameter 256 bit
python -m unittest discover -s tests -v     # jalankan semua unit test
```

Alur penggunaan ElGamal pada menu: (2) bangkitkan kunci → (3) enkripsi teks atau (5) enkripsi file dengan kunci publik → (4) dekripsi teks atau (6) dekripsi file dengan kunci privat. Mengosongkan input pada pertanyaan parameter berarti nilai dibangkitkan otomatis (acak).

BAB 6 HASIL PENGUJIAN

6.1 Reproduksi Contoh Materi

Program `demo/demo_slide.py` menghitung ulang contoh materi dan memverifikasinya dengan *assert*. Hasilnya identik dengan materi (X = 209, Y = 197, K = K' = 79; y = 461; cipherteks (1439, 74) dan (1220, 1682); nilai invers 1791 dan 1125; plainteks 'HALO').

```
########################################################################
# CONTOH 1 - Diffie-Hellman (slide 11): n=251, g=6, x=124, y=158
########################################################################

=== DIFFIE-HELLMAN KEY EXCHANGE ===
  * Parameter publik: n = 251, g = 6
  * Alice memilih x = 124 (rahasia)
      X = g^x mod n = 6^124 mod 251 = 209   --> dikirim ke Bob
  * Bob memilih y = 158 (rahasia)
      Y = g^y mod n = 6^158 mod 251 = 197   --> dikirim ke Alice
  * Alice menghitung kunci sesi
      K  = Y^x mod n = 197^124 mod 251 = 79
  * Bob menghitung kunci sesi
      K' = X^y mod n = 209^158 mod 251 = 79
  * K == K' ? True

>> Sesuai materi: X=209, Y=197, K=K'=79  [OK]

########################################################################
# CONTOH 2 - ElGamal (slide 15, 17, 18, 20): p=2273, g=3, x=243
########################################################################

=== TAHAP 1 - Pembangkitan kunci ElGamal ===
  * Bilangan prima p = 2273
  * Akar primitif g = 3
  * Kunci privat x = 243 (1 < x < p-1)
  * Kunci publik y = g^x mod p
      y = 3^243 mod 2273 = 461
  * Kunci publik  : (y, g, p) = (461, 3, 2273)
  * Kunci privat  : (x, p)    = (243, 2273)

=== TAHAP 2 - Enkripsi 'HALO' (Bob -> Alice), k1=1463, k2=2001 ===
  * Pesan 'HALO' -> 07001114 -> blok [700, 1114]
  * Enkripsi m = 700 dengan k = 1463
      a = g^k mod p     = 3^1463 mod 2273 = 1439
      b = y^k * m mod p = 461^1463 * 700 mod 2273 = 74
      cipherteks blok = (a, b) = (1439, 74)
  * Enkripsi m = 1114 dengan k = 2001
      a = g^k mod p     = 3^2001 mod 2273 = 1220
      b = y^k * m mod p = 461^2001 * 1114 mod 2273 = 1682
      cipherteks blok = (a, b) = (1220, 1682)

=== TAHAP 3 - Dekripsi (Alice) ===
  * Dekripsi (a, b) = (1439, 74)
      (a^x)^-1 = a^(p-1-x) mod p = 1439^2029 mod 2273 = 1791
      m = b * (a^x)^-1 mod p = 74 * 1791 mod 2273 = 700
  * Dekripsi (a, b) = (1220, 1682)
      (a^x)^-1 = a^(p-1-x) mod p = 1220^2029 mod 2273 = 1125
      m = b * (a^x)^-1 mod p = 1682 * 1125 mod 2273 = 1114
  * Blok [700, 1114] -> 07001114

>> Cipherteks [(1439, 74), (1220, 1682)] -> plainteks 'HALO'  [OK, sesuai materi]
```

6.2 Demo Parameter 256 Bit

Pada `demo/demo_lengkap.py`, n, g, p, dan seluruh kunci dibangkitkan otomatis dengan panjang 256 bit. Berikut cuplikan keluarannya (bilangan panjang diringkas oleh tracer menjadi awal...akhir beserta jumlah bitnya). Kunci sesi K sama di sisi Alice dan Bob, dan ElGamal memulihkan plainteks UTF-8.

```
######################################################################
# A. Diffie-Hellman dengan parameter 256 bit
######################################################################

=== TAHAP 1 - Parameter publik Diffie-Hellman ===
  * Bangkitkan prima aman n = 2q+1 (256 bit): n = 11404803232860...99524739 (256 bit)
  * Cari akar primitif g dari n: g = 2

=== DIFFIE-HELLMAN KEY EXCHANGE ===
  * Parameter publik: n = 11404803232860...99524739 (256 bit), g = 2
  * Alice memilih x = 18983833930333...20401617 (254 bit) (rahasia)
      X = g^x mod n = 2^18983833930333...20401617 (254 bit) mod 11404803232860...99524739 (256 bit) = 97476…
      ...

######################################################################
# B. Kunci sesi DH dipakai untuk cipher simetri (demo XOR-LCG)
######################################################################
  Alice mengenkripsi : b'Pesan rahasia Kelompok 11'
  Cipherteks (hex)   : 9095c650a7bc5346a866df7a25aca949798021fe2ac33b9590
  Bob mendekripsi    : b'Pesan rahasia Kelompok 11'

######################################################################
# C. ElGamal 256 bit: kunci, enkripsi & dekripsi teks UTF-8
######################################################################

=== TAHAP 1 - Pembangkitan kunci ElGamal ===
  * Bangkitkan prima aman p (256 bit): p = 10241735579207...45080707 (256 bit)
  * Akar primitif g = 2
  * Kunci privat x = 92105953500507...79126021 (256 bit) (1 < x < p-1)
  * Kunci publik y = g^x mod p
      y = 2^92105953500507...79126021 (256 bit) mod 10241735579207...45080707 (256 bit) = 28256503811152...…
  * Kunci publik  : (y, g, p) = (28256503811152...13150898 (254 bit), 2, 10241735579207...45080707 (256 bit…
  * Kunci privat  : (x, p)    = (92105953500507...79126021 (256 bit), 10241735579207...45080707 (256 bit))

  Plainteks   : Halo Dunia! Kriptografi ITS 2026
  Jumlah blok : 2 (tiap blok 31 byte)
  Blok 1: a = 38300333380874...51028334 (255 bit)
          b = 21690875627349...88614828 (254 bit)
  Blok 2: a = 41777753377642...19489704 (252 bit)
          b = 32774640355780...32489108 (255 bit)
  Hasil dekripsi : Halo Dunia! Kriptografi ITS 2026

  Enkripsi yang sama dua kali menghasilkan cipherteks berbeda (k acak):
  Blok-1 sama? False

######################################################################
# D. Serangan logaritma diskrit pada parameter KECIL vs BESAR
######################################################################
  p=2273  : x ditemukan = 243 setelah 244 percobaan (0.0000 dtk)
  p 256 bit: ruang kunci ~ 2^255 = 57896044618658...64819968 (256 bit) kemungkinan
  -> brute force tidak layak; keamanan bertumpu pada masalah logaritma diskrit.
```

6.3 Enkripsi dan Dekripsi File

File teks contoh (82 byte) dienkripsi dengan kunci publik 256 bit. Dengan ukuran blok 31 byte diperoleh 3 blok cipherteks (a, b). Setelah didekripsi dengan kunci privat, file hasil **identik byte demi byte** dengan file asli (diverifikasi dengan `cmp`).

```
$ python main.py        (menu 5: enkripsi file, menu 6: dekripsi file)
```

```
Selesai: 3 blok -> contoh/pesan_contoh.txt.enc
Selesai: 82 byte -> contoh/pesan_contoh.txt.dec
$ cmp contoh/pesan_contoh.txt contoh/pesan_contoh.txt.dec && echo FILE_IDENTIK
FILE_IDENTIK
```

6.4 Unit Test

Sebanyak 16 unit test dijalankan dengan `unittest` bawaan Python dan seluruhnya lulus (OK).

Kelompok	Yang diuji
TestMath (5)	mod_exp dibandingkan dengan pow() bawaan; gcd/extended_gcd/invers; Miller-Rabin (termasuk bilangan Carmichael 561); akar primitif; logaritma diskrit.
TestDiffieHellman (3)	Contoh materi K = 79; parameter acak 64 bit; penolakan n bukan prima dan g bukan akar primitif.
TestElGamal (5)	Contoh materi (1439,74), (1220,1682) dan 'HALO'; round-trip byte termasuk data kosong, seluruh nilai 0-255, dan byte 0x00 di depan; cipherteks bersifat probabilistik; kunci salah gagal; input tidak valid ditolak.
TestEncodingAndStorage (2)	Pemetaan huruf-angka mode slide; simpan lalu muat kunci dan cipherteks.
TestSession (1)	Cipher XOR simetris (enkripsi dua kali = plainteks).

6.5 Serangan Logaritma Diskrit: Parameter Kecil vs Besar

Pada p = 2273, penyadap yang mengetahui y = 461 menemukan x = 243 hanya dalam 244 percobaan (brute force pada `discrete_log_bruteforce`). Pada p 256 bit ruang pencarian sekitar 2^{255} kemungkinan sehingga serangan sejenis tidak layak. Hal ini menunjukkan bahwa contoh pada materi hanya untuk memahami algoritma, dan parameter praktis harus besar.

BAB 7 ANALISIS KEAMANAN DAN KETERBATASAN

- **Ukuran parameter.** Contoh kecil (p = 251 atau 2273) tidak aman. Program membangkitkan prima aman hingga ratusan bit; untuk penggunaan nyata dianjurkan p minimal 2048 bit.
- **Kerahasiaan k.** Nilai k pada ElGamal harus acak dan tidak boleh dipakai ulang; bila dipakai ulang untuk dua pesan, penyerang dapat menghubungkan kedua pesan. Program memilih k baru setiap blok dari `SystemRandom`.
- **Serangan man-in-the-middle.** Diffie-Hellman murni hanya aman terhadap penyadap pasif. Penyerang aktif dapat menyisipkan nilai X/Y palsu karena tidak ada autentikasi; solusinya menggabungkan tanda tangan digital atau sertifikat (tidak dicakup program ini).
- **Tanpa padding.** ElGamal pada program ini bersifat *malleable* (cipherteks dapat dimodifikasi sehingga plainteks berubah secara terprediksi) dan tidak CCA-secure. Sistem nyata memakai skema padding atau pendekatan hibrida.
- **Cipher simetri demo.** XOR dengan keystream LCG pada `session.py` hanya ilustrasi pemakaian kunci sesi; untuk keamanan nyata gunakan cipher modern seperti AES-GCM.

- **Kinerja.** Implementasi pure Python dan pencetakan langkah membuat file besar diproses lambat (tiap blok membutuhkan dua eksponensiasi modular). Ekspansi cipherteks sekitar dua kali lipat atau lebih karena pasangan (a, b) ditulis dalam bilangan desimal.
- **Uji keprimaan probabilistik.** Miller-Rabin 40 putaran memiliki peluang salah yang dapat diabaikan, tetapi bukan bukti keprimaan yang pasti.

BAB 8 KESIMPULAN

Program yang dibuat berhasil mengimplementasikan pertukaran kunci Diffie-Hellman dan algoritma ElGamal tanpa library kriptografi jadi, dengan setiap tahapan (parameter publik, nilai rahasia dan publik, kunci sesi; pembangkitan kunci, enkripsi, dekripsi) dipisahkan dalam fungsi tersendiri dan dapat ditampilkan langkah demi langkah. Hasil perhitungan identik dengan contoh materi (K = 79 dan 'HALO'), program bekerja pada parameter 256 bit untuk teks UTF-8 maupun file (dekripsi memulihkan data asli), dan 16 unit test lulus. Keamanan kedua algoritma bertumpu pada sulitnya logaritma diskrit; demonstrasi serangan memperlihatkan parameter kecil dapat dipecahkan seketika sedangkan parameter besar tidak. Pengembangan selanjutnya dapat berupa autentikasi pada Diffie-Hellman, padding pada ElGamal, dan penggunaan cipher simetri modern untuk kunci sesi.

DAFTAR PUSTAKA

[1] Aumasson, J.-P. (2017). *Serious Cryptography: A Practical Introduction to Modern Encryption*. San Francisco: No Starch Press.

[2] Munir, R. (2019). *Kriptografi* (Edisi Kedua). Bandung: Penerbit Informatika.

[3] Materi kuliah Kriptografi, *Pertemuan ke-9: ElGamal dan Diffie-Hellman* (Diffie-Hellman Key Exchange and ElGamal Algorithm, Week 9).

[4] Indrawanti, A. S. (2022). *Kriptografi - Algoritma Pertukaran Kunci Diffie Hellman and ElGamal* [Video]. YouTube. https://www.youtube.com/watch?v=5fctvHaqoWI

[5] Diffie, W., & Hellman, M. (1976). New Directions in Cryptography. *IEEE Transactions on Information Theory, 22*(6), 644-654.

[6] ElGamal, T. (1985). A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms. *IEEE Transactions on Information Theory, 31*(4), 469-472.